

南京信息工程大学 实验（实习）报告

实验（实习）名称 循环码编码 日期 2026.5.26 得分

指导教师 崔琦 班级 24 奇安信 1 班 姓名 孙涛 学号 202483950014

一、实验目的

本次实验的目的包括理解循环码的基本定义及其循环移位封闭性质；掌握生成多项式 $g(x)$ 与循环码编码之间的关系；掌握二进制循环码的系统编码方法并编程实现；编程实现循环码的编码过程；加深对循环码在差错控制编码中的应用理解。

二、实验原理

本次实验须掌握的原理有以下几方面，首先是对循环码定义和概念的掌握，循环码最本质的特征是：码字向量的任何循环移位仍然是该码字集合中的一个码字。即对于通信发送方想传输的 k 位信息位加上 r 位校验位组成的 n 长码字 c ， c 进行原地循环左移或循环右移后的结果仍在 c 中。对于 n 长的码字，可以讲码字 C 表示为多项式 $c_i \cdot x^i$ 求和。

码字 C 如何得到，对于想要发送的信息多项式 $m(x)$ ，乘上给定的生成多项式 $g(x)$ ，即可得到 $c(x)=m(x)*g(x)$ ，而该式的另一种写法即系统化编码 $c(x)=x^{(n-k)}*m(x)+r(x)$ 与其等价且更常用，其中 $r(x)$ 是余数系统化编码的好处在于对于接收方看到得到的 cx 直接观察前 k 位即可得到信息，无需再做 cx/gx 的运算。为什么这两个式子是等价的，因为 $x^{\{n-k\}}*m(x)=q(x)*g(x)+r(x)$ ，而在 $GF(2)$ 运算中， $+r(x)$ 可直接移到等式左边，即 $x^{\{n-k\}}*m(x)+r(x)$ 也可整除 $g(x)$ 。系统化编码实际上在做的事情就是，把 k 位信息位的右边加上 r 位 0，然后用此时得到的这个 n 位数作为被除数，除数为 $g(x)$ ，模 2 除法得到的 r 位余数再取代尾部的 r 位 0，就得到可以发送出去的码字 C 。这一过程在计算机网络中的 CRC 循环冗余校验就已经实际应用。

完成了编码后，接收方对于接收到的 n 位码字只需除以约定好的 $g(x)$ 得到伴随式 S ，若 S 为 0 说明无错，若 S 不为 0 说明传输过程出错。在模 $g(x)$ 运算下伴随式 $S=c(x)+e(x)$ ，那么通过观察 ex 上哪一位为 1 就可以知道哪位出错。

为什么对于一个合法码字 c_1 进行循环移位后得到的码字任是合法码字，只需证明左移 1 位后的码字任合法就可以推导出这个结论。而左移 1 位其实就是拿 $c_1 * x$ ，已知 c_1 能整除 $g(x)$ ，所以 $c_1 * x$ 当然也能整除 $g(x)$ 。

三、实验内容

实验的主要内容包括：实现 GF(2) 多项式除法，实现循环码系统编码，验证循环码性质

四、实验代码和结果截图

实验的核心步骤分 3 步，第一步是实现模 2 除法，具体实现代码如下（本次实验全程规定生成多项式为 $g(x)=X^3+X+1$ ）：

```
// GF(2) 多项式除法：
// dividend 是被除多项式，divisor 是除式
// remainder 返回余数，长度为 divisor_len - 1
void gf2_divide(const int dividend[], int dividend_len,
                const int divisor[], int divisor_len,
                int remainder[]) {
    int temp[MAXN];
    memset(temp, 0, sizeof(temp));

    for (int i = 0; i < dividend_len; i++) {
        temp[i] = dividend[i];
    } // 初始化 temp 为 dividend 的副本

    // 模 2 长除法：遇到 1 就异或一次除式
    for (int i = 0; i <= dividend_len - divisor_len; i++) {
        if (temp[i] == 1) {
            for (int j = 0; j < divisor_len; j++) {
                temp[i + j] ^= divisor[j]; // 异或除式
            }
        }
    }
}
```

```

    }
}

// 最后 divisor_len-1 位就是余数
int rem_len = divisor_len - 1;
int start = dividend_len - rem_len;
for (int i = 0; i < rem_len; i++) {
    remainder[i] = temp[start + i];
}
}

```

第二个核心步骤是对于给定的 4 位信息位实现循环码编码得到 7 位的码字，代码实现如下：

```

// 系统循环码编码
// msg: 信息位，长度 k
// g: 生成多项式，长度 r+1
// codeword: 输出码字，长度 n = k + r
void encode_systematic(const int msg[], int k,
                       const int g[], int g_len,
                       int codeword[]) {
    int r = g_len - 1;
    int dividend[MAXN];
    int remainder[MAXN];

    // 1) 信息位后补 r 个 0
    for (int i = 0; i < k; i++) {
        dividend[i] = msg[i];
    }
    for (int i = k; i < k + r; i++) {
        dividend[i] = 0;
    }
}

```

// 2) 计算余数

```
gf2_divide(dividend, k + r, g, g_len, remainder);
```

// 3) 系统码字 = 原信息位 + 余数

```
for (int i = 0; i < k; i++) {  
    codeword[i] = msg[i];  
}  
for (int i = 0; i < r; i++) {  
    codeword[k + i] = remainder[i];  
}  
}
```

第三个核心步骤是检验生成的码字以及对码字进行循环移位后的码字是否合法，代码如下（除法运算步骤第 1 步中已给出，此处不再重复）：

// 判断余数是否全 0

```
int is_zero_remainder(const int rem[], int len) {  
    for (int i = 0; i < len; i++) {  
        if (rem[i] != 0) return 0;  
    }  
    return 1;  
}
```

最终测试结果如下图：

问题 输出 调试控制台 终端

```
Active code page: 65001
PS C:\Users\23059\Desktop\信息论\cycliccode> gcc cyclic_code.cpp -o cyclic_code.exe
PS C:\Users\23059\Desktop\信息论\cycliccode> ./cyclic_code.exe
===== 循环码编码实验 =====
码长 n = 7
信息位 k = 4
监督位 r = 3
生成多项式 g(x) = [1, 0, 1, 1]
请输入 4 位信息位 (用空格分开, 例如 1 0 1 1):
1 0 1 1
原始信息位 = [1, 0, 1, 1]
编码得到的循环码码字 = [1, 0, 1, 1, 0, 0, 0]
该码字是否能被 g(x) 整除: True
循环右移 1 位后的码字 = [0, 1, 0, 1, 1, 0, 0]
循环移位后是否仍为合法码字: True
PS C:\Users\23059\Desktop\信息论\cycliccode> 
```

五、实验心得

实验让我深入理解了循环码的编码原理及其实际应用。通过本次实验，学习了“补零+模 2 除法”的过程如何利用 $g(x)$ 的因子特性来构建校验序列。了解了通过计算伴随式来定位错误位置的 s 过程，也了解了在实际的应用如计算机网络通信中的 CRC 循环冗余校验是如何实现的，以及发现错误后的反应机制。